

The function that will handle all three events will be the `fcopy_tags()` function; this function will be invoked by the `tcopy_tags` trigger. The `fcopy_tags()` function using the `TG_OP` variable will be able to discriminate between the `INSERT`, `UPDATE`, and `DELETE` events.

Let's start by writing the `fcopy_tags()` function to handle the `INSERT` event:

```

forumbd=> CREATE OR REPLACE FUNCTION fcopy_tags() RETURNS trigger as
$$
BEGIN
IF TG_OP = 'INSERT' THEN
    IF lower(substring(NEW.tag from 1 for 1)) = 'a' THEN
        insert into new_a_tags(pk,tag,parent) values (NEW.pk,NEW.tag,NEW.
parent);
    ELSIF lower(substring(NEW.tag from 1 for 1)) = 'b' THEN
        insert into new_b_tags(pk,tag,parent) values (NEW.pk,NEW.tag,NEW.
parent);
    END IF;
    RETURN NEW;
END IF;
END;
$$
LANGUAGE 'plpgsql';
CREATE FUNCTION

forumbd=> CREATE TRIGGER tcopy_tags_ins BEFORE INSERT on new_tags FOR EACH
ROW EXECUTE PROCEDURE fcopy_tags();
CREATE TRIGGER

```

Now let's see if, for the `INSERT` event, this code works:

```

forumbd=> insert into new_tags (pk,tag,parent) values (1,'operating
systems',NULL);
INSERT 0 1
forumbd=> insert into new_tags (pk,tag,parent) values (2,'alpine
linux',1);
INSERT 0 1
forumbd=> insert into new_tags (pk,tag,parent) values (3,'bsd unix',1);
INSERT 0 1

forumbd=> select * from new_a_tags ;

```

```

pk |      tag      | parent
----+-----+-----
  2 | alpine linux |      1
(1 row)

forumbd=> select * from new_b_tags ;
pk |   tag   | parent
----+-----+-----
  3 | bsd unix |      1
(1 row)

forumbd=> select * from new_tags ;
pk |      tag      | parent
----+-----+-----
  1 | operating systems |
  2 | alpine linux      |      1
  3 | bsd unix          |      1
(3 rows)

```

It is clear that it works!

Next, let's handle the DELETE event. The things we need to do are the following:

- Add some lines of code to the function to manage the DELETE operation.
- Add a new trigger that is able to handle the DELETE event.

The function becomes as follows:

```

forumbd=> CREATE OR REPLACE FUNCTION fcopy_tags() RETURNS trigger as
$$
BEGIN
IF TG_OP = 'INSERT' THEN
    IF lower(substring(NEW.tag from 1 for 1)) = 'a' THEN
        insert into new_a_tags(pk,tag,parent)values (NEW.pk,NEW.tag,NEW.
parent);
    ELSEIF lower(substring(NEW.tag from 1 for 1)) = 'b' THEN
        insert into new_b_tags(pk,tag,parent)values (NEW.pk,NEW.tag,NEW.
parent);
    END IF;
RETURN NEW;

```

```

END IF;
IF TG_OP = 'DELETE' THEN
    IF lower(substring(OLD.tag from 1 for 1)) = 'a' THEN
        DELETE FROM new_a_tags WHERE pk = OLD.pk;
    ELSIF lower(substring(OLD.tag from 1 for 1)) = 'b' THEN
        DELETE FROM new_b_tags WHERE pk = OLD.pk;
    END IF;
    RETURN OLD;
END IF;
END;
$$
LANGUAGE 'plpgsql';
CREATE FUNCTION

```

This piece of code was added:

```

IF TG_OP = 'DELETE' THEN
    IF lower(substring(OLD.tag from 1 for 1)) = 'a' THEN
        DELETE FROM new_a_tags WHERE pk = OLD.pk;
    ELSIF lower(substring(OLD.tag from 1 for 1)) = 'b' THEN
        DELETE FROM new_b_tags WHERE pk = OLD.pk;
    END IF;
    RETURN OLD;
END IF;

```

This piece of code deletes the data in the a_tags and b_tags tables if the record to be deleted begins with the letter a or b. Now we have to create a new trigger that is able to handle DELETE events:

```

forumbd=> CREATE TRIGGER tcopy_tags_del
AFTER DELETE on new_tags FOR EACH ROW EXECUTE PROCEDURE fcopy_tags();
CREATE TRIGGER

```

The trigger is executed AFTER DELETE; in this case, it would have made no difference if we created the TRIGGER BEFORE or AFTER INSERT functions. Let's see if this trigger on the DELETE event works:

```

forumbd=> delete from new_tags where pk=2;
DELETE 1
forumbd=> delete from new_tags where pk=3;
DELETE 1

```

```

forumbd=> select * from new_a_tags ;
pk | tag | parent
----+-----+-----
(0 rows)

forumbd=> select * from new_b_tags ;
pk | tag | parent
----+-----+-----
(0 rows)

forumbd=> select * from new_tags ;
pk |          tag          | parent
----+-----+-----
  1 | operating systems |
(1 row)

```

As we can see, the TRIGGER works.

For the last step, we need to manage the UPDATE event. Let's write the function and the triggers as a full version from scratch. Again, let's bring our environment back to the initial conditions:

```

forumbd=> truncate new_tags ;
TRUNCATE TABLE
forumbd=> truncate new_a_tags ;
TRUNCATE TABLE
forumbd=> truncate new_b_tags ;
TRUNCATE TABLE

forumbd=> insert into new_tags (pk,tag,parent) values (1,'operating
systems',NULL),(2,'alpine linux',1),(3,'bsd unix',1);
INSERT 0 3

```

Now we can write the complete code for the UPDATE event:

```

forumbd=> CREATE OR REPLACE FUNCTION fcopy_tags() RETURNS trigger as
$$
BEGIN
IF TG_OP = 'INSERT' THEN
    IF lower(substring(NEW.tag from 1 for 1)) = 'a' THEN

```

```

        insert into new_a_tags(pk,tag,parent)values (NEW.pk,NEW.tag,NEW.
parent);
        ELIF lower(substring(NEW.tag from 1 for 1)) = 'b' THEN
            insert into new_b_tags(pk,tag,parent)values (NEW.pk,NEW.tag,NEW.
parent);
        END IF;
        RETURN NEW;
    END IF;
IF TG_OP = 'DELETE' THEN
    IF lower(substring(OLD.tag from 1 for 1)) = 'a' THEN
        DELETE FROM new_a_tags WHERE pk = OLD.pk;
    ELIF lower(substring(OLD.tag from 1 for 1)) = 'b' THEN
        DELETE FROM new_b_tags WHERE pk = OLD.pk;
    END IF;
    RETURN OLD;
END IF;
IF TG_OP = 'UPDATE' THEN
    IF (lower(substring(OLD.tag from 1 for 1)) in( 'a','b') ) THEN
        DELETE FROM new_a_tags WHERE pk=OLD.pk;
        DELETE FROM new_b_tags WHERE pk=OLD.pk;
        DELETE FROM new_tags WHERE pk = OLD.pk;
        INSERT into new_tags(pk,tag,parent) values (NEW.pk,NEW.tag,NEW.
parent);
    END IF;
    RETURN NEW;
END IF;
END;
$$
LANGUAGE 'plpgsql';
CREATE FUNCTION

forumdb=> CREATE TRIGGER tcopy_tags_upd
    AFTER UPDATE on new_tags FOR EACH ROW EXECUTE PROCEDURE fcopy_tags();
CREATE TRIGGER

```

In this case, the trigger must be defined with `AFTER UPDATE` and not with `BEFORE UPDATE` because in the `UPDATE` section, we have the instruction `DELETE FROM new_tags WHERE pk = OLD.pk`; if the trigger had been defined with `BEFORE UPDATE`, we would have had an error because we would have attempted to delete a record reserved for `UPDATE`.

Let's see if the complete function works:

```

forumdb=> select * from new_tags;
pk |      tag      | parent
----+-----+-----
  1 | operating systems |
  2 | alpine linux    |      1
  3 | bsd unix        |      1
(3 rows)

forumdb=> select * from new_a_tags;
pk |   tag   | parent
----+-----+-----
  2 | alpine linux |      1
(1 row)

forumdb=> select * from new_b_tags;
pk |   tag   | parent
----+-----+-----
  3 | bsd unix |      1
(1 row)

forumdb=> update new_tags set tag='apple dos' where pk=3;
UPDATE 1

forumdb=> select * from new_a_tags;
pk |   tag   | parent
----+-----+-----

```

```

  2 | alpine linux |      1
  3 | apple dos   |      1
(2 rows)

forumdb=> select * from new_tags;
 pk |      tag      | parent
-----+-----+-----
  1 | operating systems |
  2 | alpine linux    |      1
  3 | apple dos      |      1
(3 rows)

```

As this shows, the trigger approach works. In this section, we have seen how to modify events that are **Data Manipulation Level (DML)** through the use of rules and triggers. In the next section, we will see how it is also possible to intercept and modify events related to DDL operations, using event triggers.

Event triggers

Rules and triggers act as DML statements, which means they are triggered by something that changes the data but not the data layout or the table properties. PostgreSQL provides so-called *event triggers*, which are particular triggers that fire on **DDL** statements. The purpose of the event trigger, therefore, is to manage and react to events that will change the data structure rather than the data content. Triggers can be used in many ways to enforce specific policies across your databases.

Once fired, an event trigger receives an *event* and a *command tag*, both of which are useful for introspection and providing information about what fired the trigger. In particular, the *command tag* contains a description of the command (for example, CREATE or ALTER), while the *event* contains the category that fired the trigger – in particular, the following:

- `ddl_command_start` and `ddl_command_end` indicate, respectively, the beginning and the completion of the DDL command.
- `sql_drop` indicates that a DROP command is near completion.
- `table_rewrite` indicates that a full table rewrite is about to begin.

As with DML triggers, there are particular commands to create, delete, and modify an event trigger:

- `CREATE EVENT TRIGGER` to add a new event trigger
- `DROP EVENT TRIGGER` to delete an existing trigger
- `ALTER EVENT TRIGGER` to modify an existing trigger

Here is the synopsis for the creation of a new event trigger:

```
CREATE EVENT TRIGGER name
ON event
[ WHEN filter_variable IN (filter_value [, ... ]) [ AND ... ] ]
EXECUTE { FUNCTION | PROCEDURE } function_name()
```

Similar to their DML counterpart triggers, event triggers are associated with a mnemonic name and a function to execute once they are fired. However, unlike ordinary triggers, event triggers do not specify which table they are attached to; in fact, event triggers are not related to any particular table but, rather, to DDL commands.

Event triggers must be created by the database administrator and have a database scope, meaning they live and act in the database they have been defined in.

There are a couple of special functions that can help developers perform introspection within an event trigger to understand the exact event that fired the trigger. The most important functions are as follows:

- `pg_event_trigger_commands()`, which returns a tuple for every command that was executed during the DDL statement.
- `pg_event_trigger_dropped_objects()`, which reports a tuple for every dropped object within the same DDL statement.

Along with the preceding utility functions, it is important to carefully read the event trigger documentation to understand when a command will fire an event trigger or not. Explaining event triggers in further detail is out of the scope of this section; instead, we will look at a practical example in the following section. For more information about event triggers, please refer to the official documentation or the Packt book *PostgreSQL 11 Server-Side Programming*.

An example of an event trigger

In order to better understand how event triggers work, let's build a simple example of a trigger that prevents any `ALTER TABLE`-like commands in a database.

The first step is to define a function that will be executed once the trigger has been fired; such a function needs to inspect the DDL statement properties to understand whether it has been invoked by means of an `ALTER TABLE` command. The introspection is done using the `pg_event_trigger_ddl_commands()` special function, which returns a tuple for every DDL statement executed within the same command. Such tuples contain a field named `command_tag`, which reports the command group (uppercase), and `object_type`, which reports the object type (lowercase) that the DDL statement has been executed against. The function must return a trigger type, specifically an event trigger type; therefore, the function can be defined as follows:

```
forumdb=> CREATE OR REPLACE FUNCTION
f_avoid_alter_table()
RETURNS EVENT_TRIGGER
AS
$code$
DECLARE
event_tuple record;
BEGIN
    FOR event_tuple IN SELECT * FROM          pg_event_trigger_ddl_
commands() LOOP
        IF event_tuple.command_tag = 'ALTER TABLE' AND event_tuple.object_
type = 'table' THEN
            RAISE EXCEPTION 'Cannot execute an ALTER TABLE!';
        END IF;
    END LOOP;
END
$code$
LANGUAGE plpgsql;
CREATE FUNCTION
```

As you can see, if the function discovers that the executed command has an `ALTER TABLE` tag and a `table` object type, it raises an exception, causing the whole statement to fail.

Once the function is in place, it is possible to attach it to an event trigger, but because event triggers handle DDL statements, only superusers can create an event trigger; so first, let's connect to the forum database as a superuser:

```
forumdb=> \q
postgres@learn_postgresql:~$ psql forumdb
```

```
psql (15.2 (Debian 15.2-1.pgdg110+1))
Type "help" for help.

forumdb=#
```

And then let's execute:

```
forumdb=# CREATE EVENT TRIGGER tr_avoid_alter_table ON ddl_command_end
EXECUTE FUNCTION forum.f_avoid_alter_table();
CREATE EVENT TRIGGER
```

Remember that we have connected as a postgres user to the database forumdb, so we have to specify the schema in which the postgres user can find the `f_avoid_alter_table()` function. At this point, the trigger is active, and the function will be fired for every DDL command once the system approaches the end of a command.

It is now possible to test the trigger and see whether a user is allowed to execute `ALTER TABLE`:

```
forumdb=> ALTER TABLE tags ADD COLUMN thumbs_up int DEFAULT 0;
ERROR:  Cannot execute an ALTER TABLE!
CONTEXT:  PL/pgSQL function f_avoid_alter_table() line 9 at RAISE
```

As we can see, an exception is raised as soon as the `ALTER TABLE` command is executed, and we have this behavior for not only the non-superuser user (as we've just seen) but also the superuser; this is because the event trigger we wrote intercepts the alter table command and modifies its behavior:

```
forumdb=# ALTER TABLE forum.tags ADD COLUMN thumbs_up int DEFAULT 0;
ERROR:  Cannot execute an ALTER TABLE!
CONTEXT:  PL/pgSQL function forum.f_avoid_alter_table() line 9 at RAISE
```

While event triggers can be used, as in the preceding example, to prevent users from executing particular commands, a better strategy is to avoid inappropriate command executions by means of permissions whenever possible. Event triggers are complex and are used to provide support for things such as logical replication, auditing, and other infrastructures.

Summary

In this chapter, we covered the topic of triggers and rules. We explored rules and triggers using some identical examples. We established that rules are simple event handlers and triggers are complex event handlers.